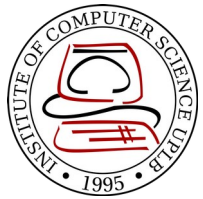
CMSC 137

Data Communications and Networking

University of the Philippines
LOS BAÑOS

Lecture Module
21 - Routing



Module 21: Routing

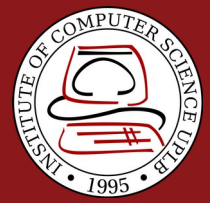
Rozano S. Maniaol
Lecturer

Introduction

The goal of the network layer is to deliver a datagram from its source to its destination or destinations

Terms:

- Routing – process of selecting a path for traffic in a network or across multiple networks
  - Unicast Routing – if a datagram is destined for only one destination
  - Multicast Routing – if a datagram is destined for several destinations
- Forwarding – the way a packet is delivered to the next station



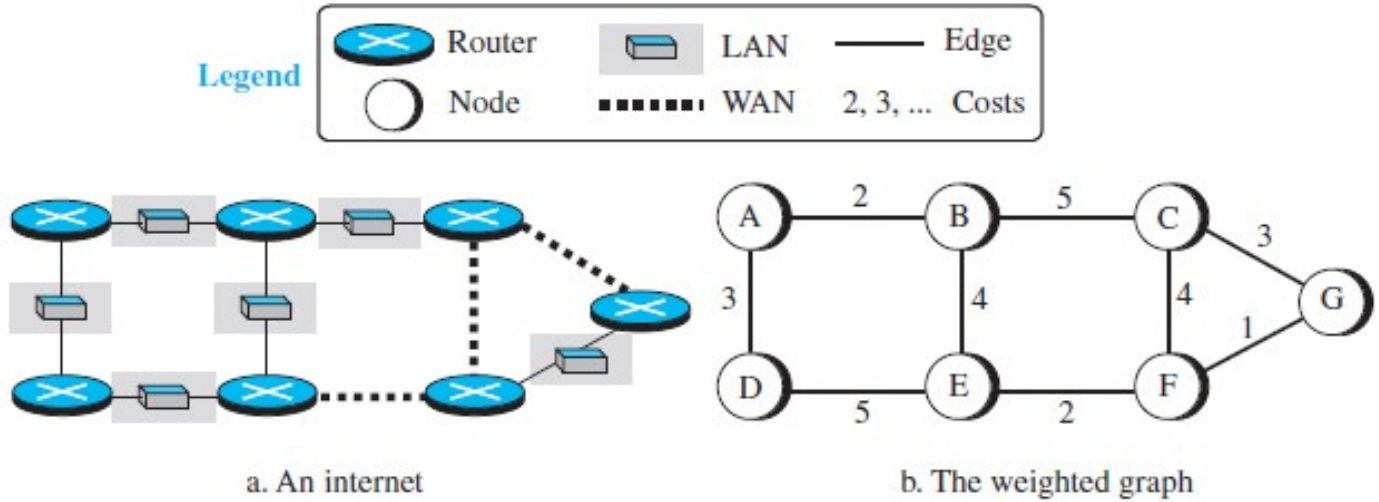
Unicast Routing

Unicast Routing

- A packet is routed, hop by hop, from its source to its destination by the help of forwarding tables
- Routing a packet from its source to destination means routing the packet from a source router (the default router of the source host) to a destination router (the router connected to the destination network)
- To find the best route, an internet can be modeled as a graph
  - Actually modeled as a weighted graph

Least Cost Routing

The source router chooses a route to the destination router in such a way that the total cost for the route is the least cost among all possible routes

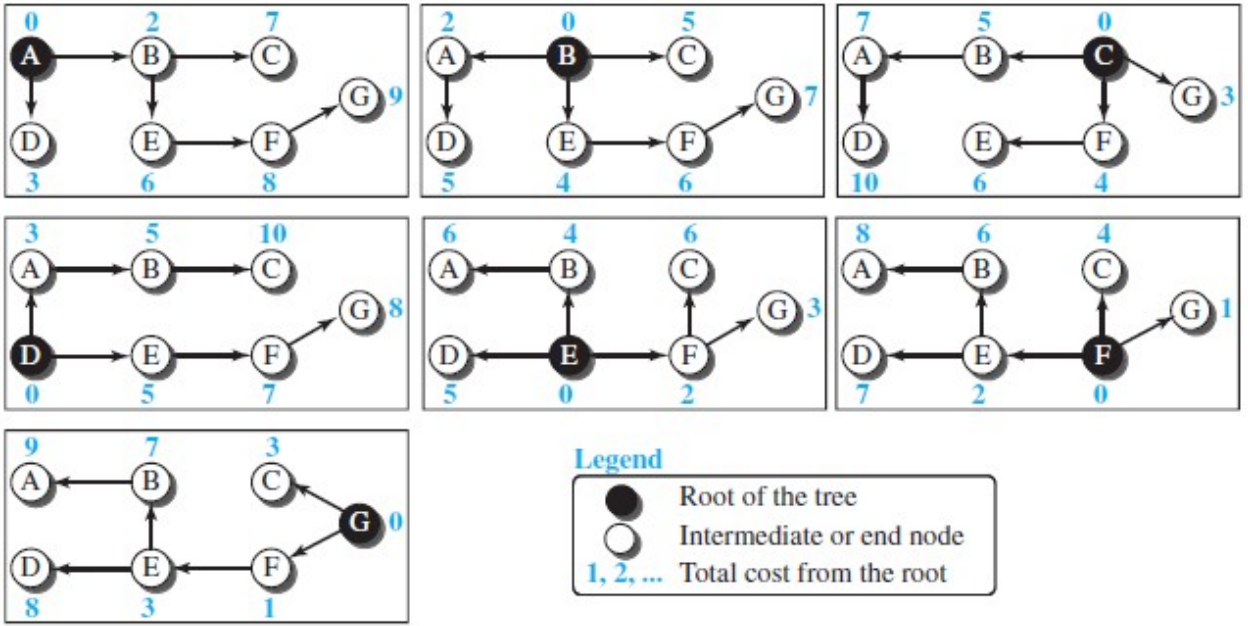


a. An internet

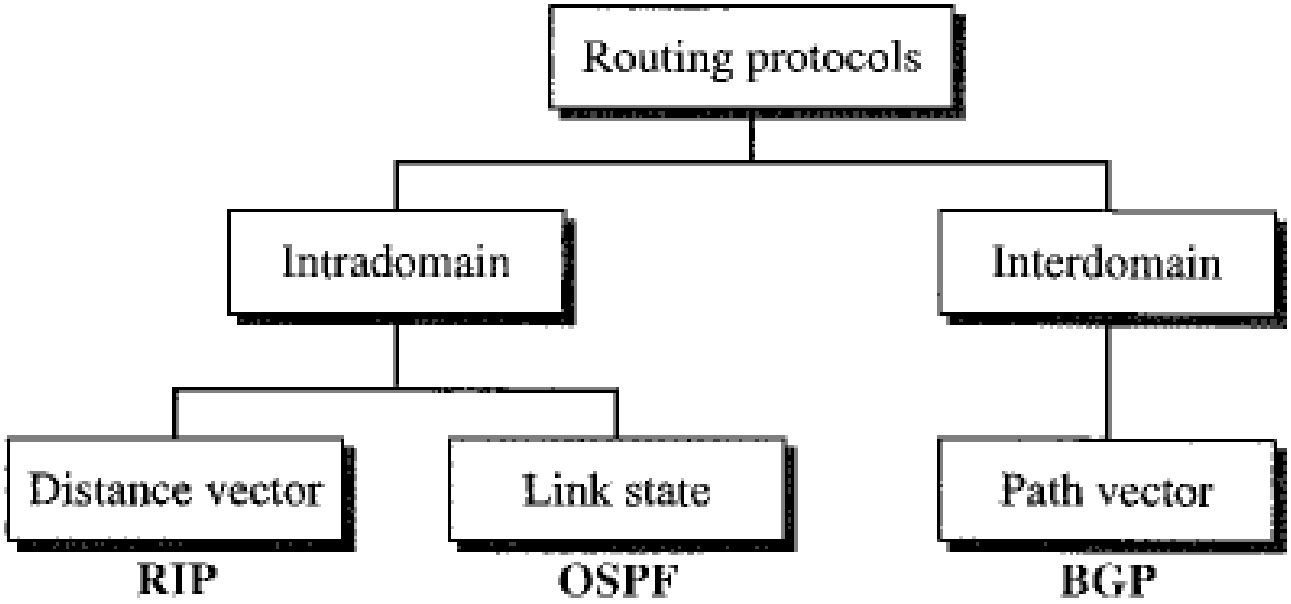
b. The weighted graph

Least Cost Tree

Tree with the source router as the root that spans the whole graph and in which the path between the root and any other node is the shortest



Routing Algorithm



Distance-Vector Routing

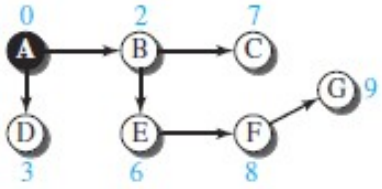
- Each node creates its own least-cost tree with the rudimentary information it has about its immediate neighbors
- The incomplete trees are exchanged between immediate neighbors to make the trees more and more complete and to represent the whole internet
- Bellman-Ford equation – used to find the least cost between a source node x, and a destination node y, through some intermediary nodes (a, b, c,…) when the costs between the source and the intermediary nodes and the least costs between the intermediary nodes and destination are given

$$D_{xy} = min\{(c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), ...\}$$

D_{ij} – shortest distance
c_{ij} – cost between nodes i and j

Distance Vector

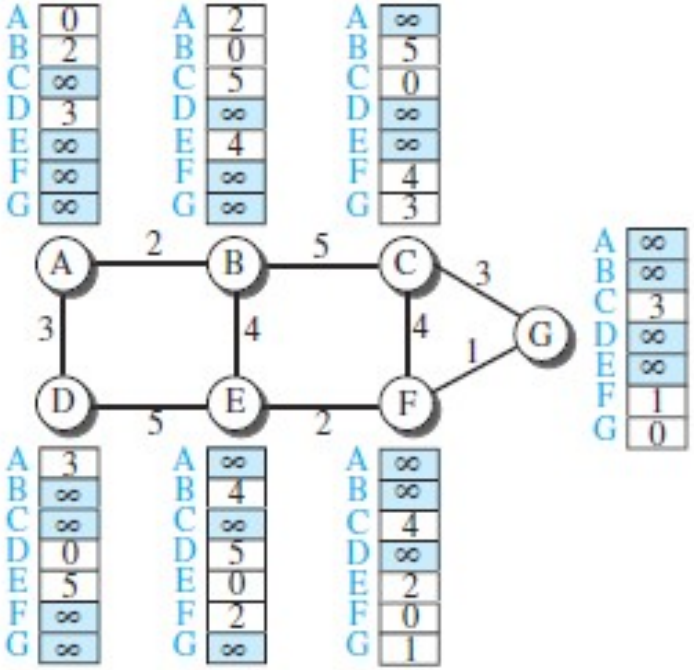
- One-dimensional array to represent the tree
- The name defines the root, the indexes defines the destinations and the values of each cell defines the least cost from the root to the destination
- A distance vector does not give the path to the destination, only the least costs to the destination



a. Tree for node A

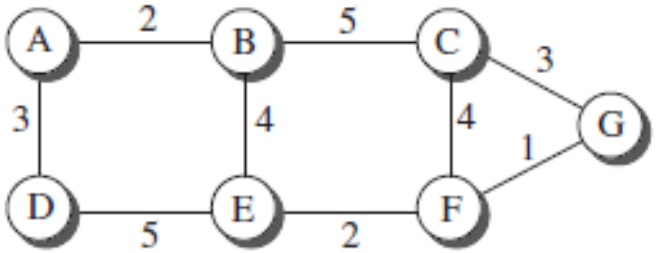
	A
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A



Link-State Routing

- This method uses the term link-state to define the characteristic of a link (edge) that represents a network in the internet
- To create a least-cost tree, each node needs a complete map of the network
- The collection of states for all links is called the link-state database (LSDB)



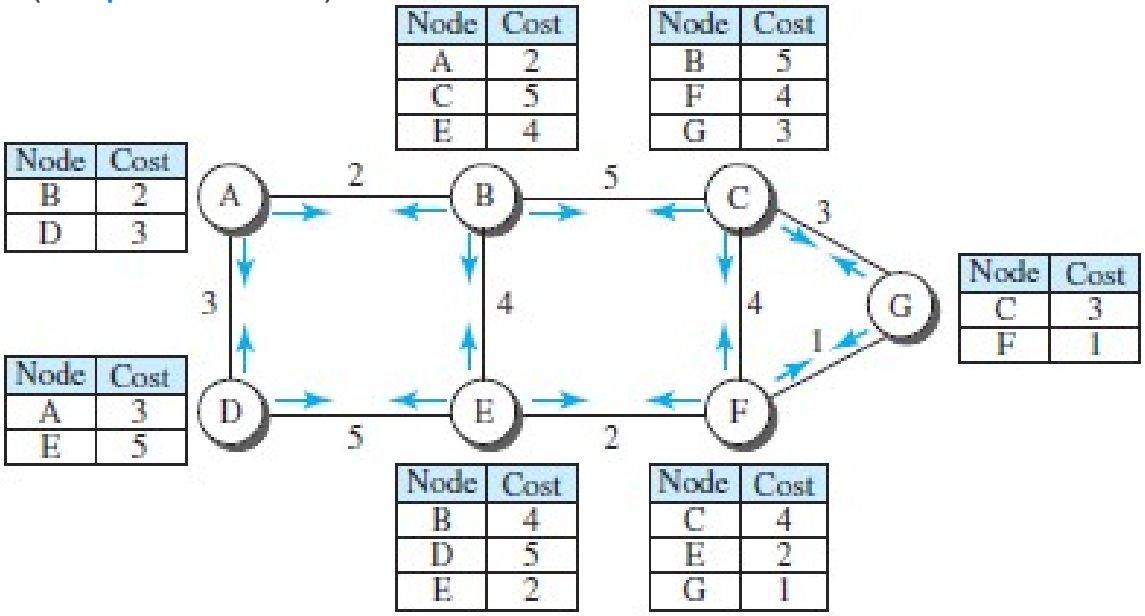
a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

Link-State Routing

- Flooding – each node can send some greeting messages to all its immediate neighbors to collect two pieces of information for each neighboring node: the identity of the node and the cost of the link (LS packet/LSP)

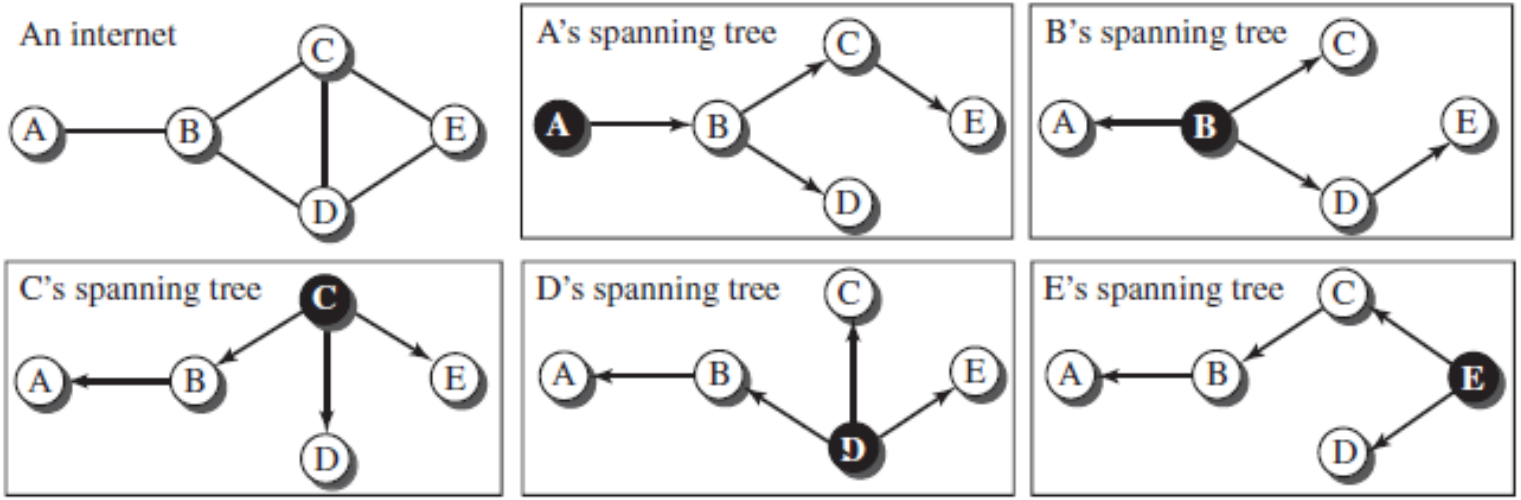


Link-State Routing

- Formation of Least-Cost Trees
  - Using the shared LSDB, each node runs the Dijkstra Algorithm
    1) The node chooses itself as the root of the tree, creating a tree with a single node, and sets the total cost of each node on the information in the LSDB
    2) The node selects one node, among all nodes not in the tree, which is closest to the root, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed
    3) The node repeats step 2 until all nodes are added to the tree

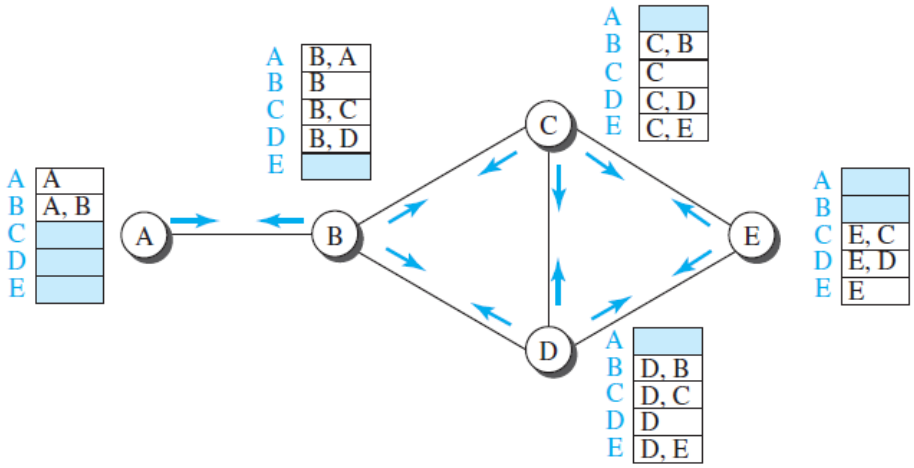
Path-Vector Routing

- The best path is determined by the source using the policy it imposes on the route, i.e. the source controls the path
- The path is also determined by the best spanning tree. It is not the least-cost tree; it is the tree determined by the source when it imposes its own policy

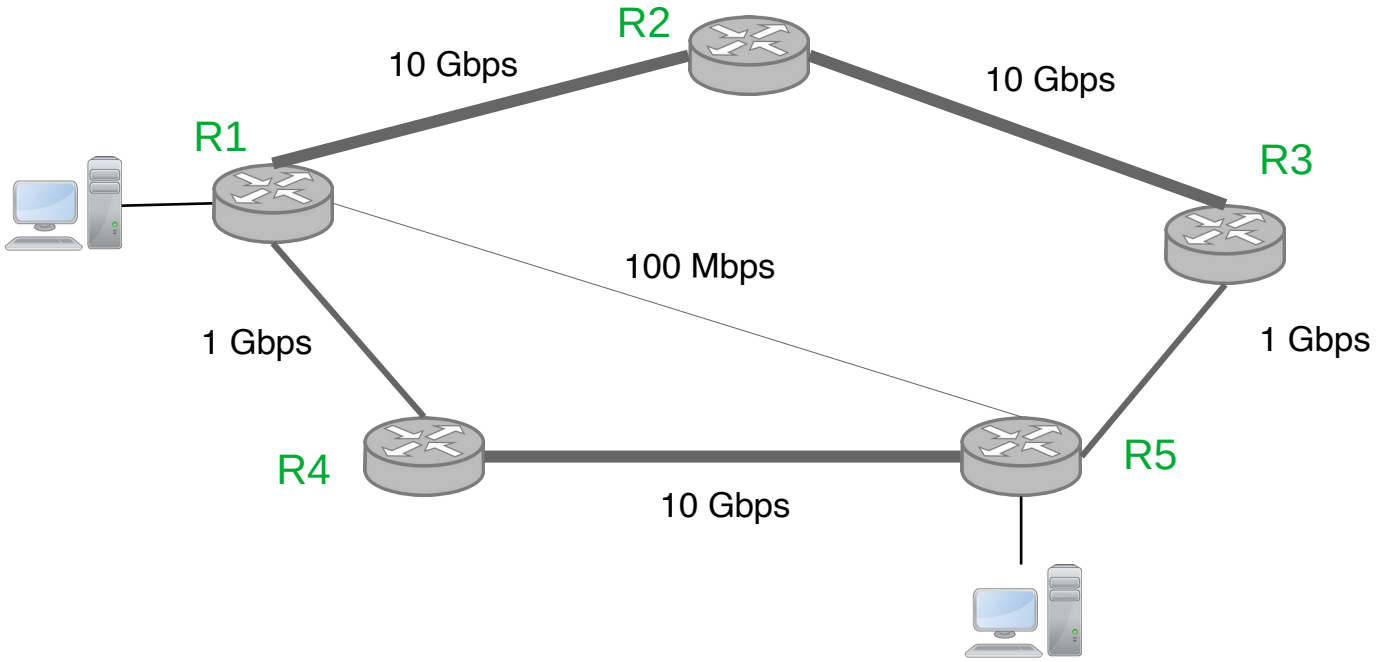


Path-Vector Routing

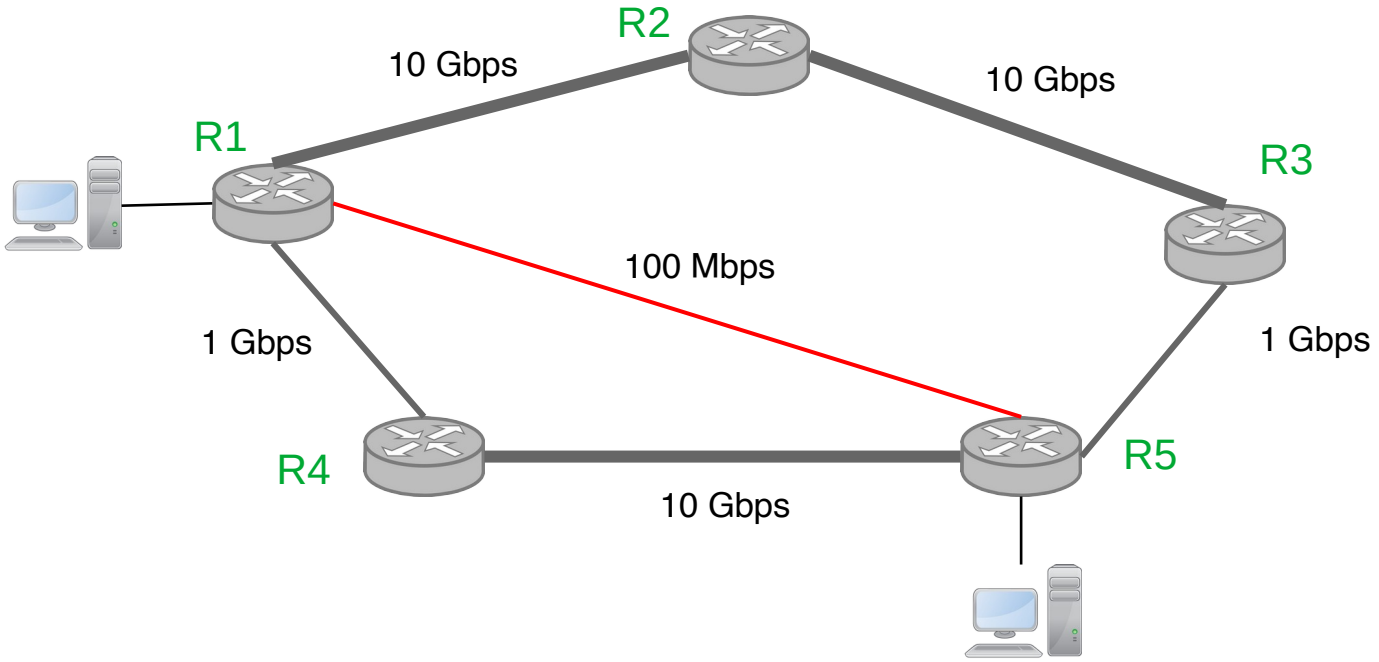
- Spanning trees are made by each node gradually and asynchronously
  - When a node is booted, it creates a path vector based on the information it can obtain about its immediate neighbor
  - Each node sends it to all it immediate neighbors.
  - Each node, when it receives a path vector, updates it path vector by applying its own policy instead of look for the least cost
  - The policy is defined by selecting the best of multiple paths



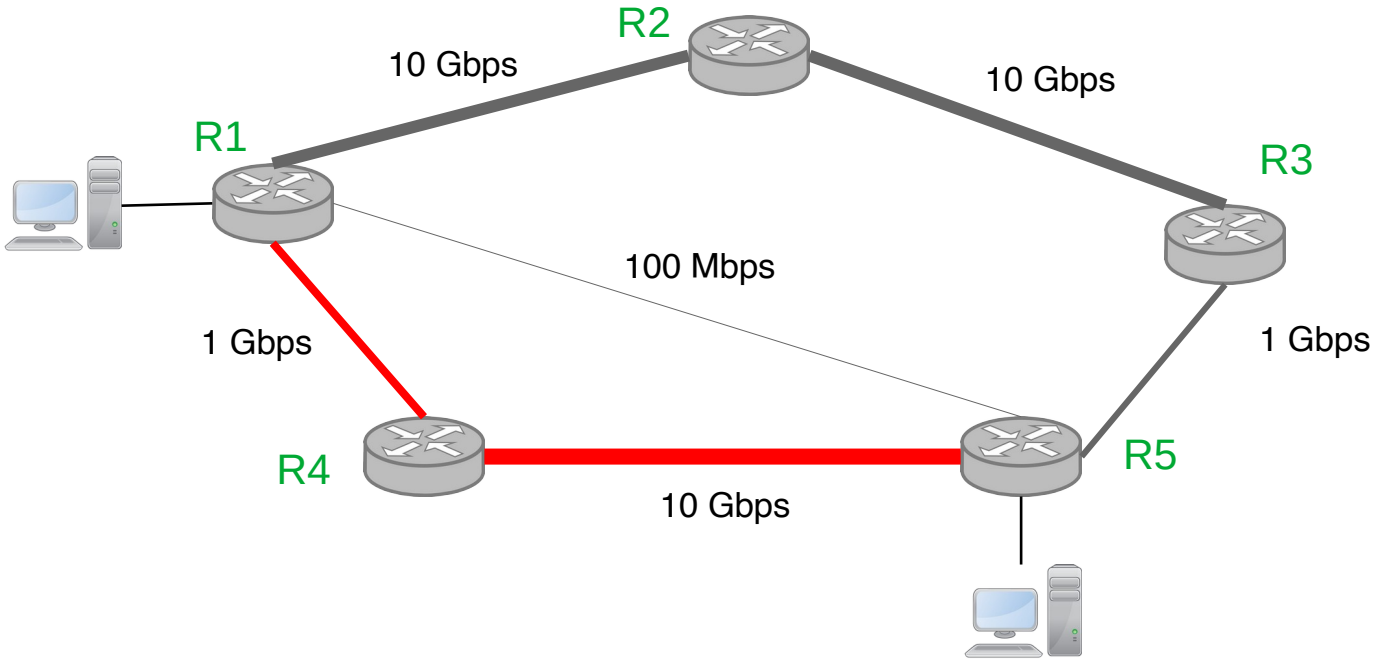
Example



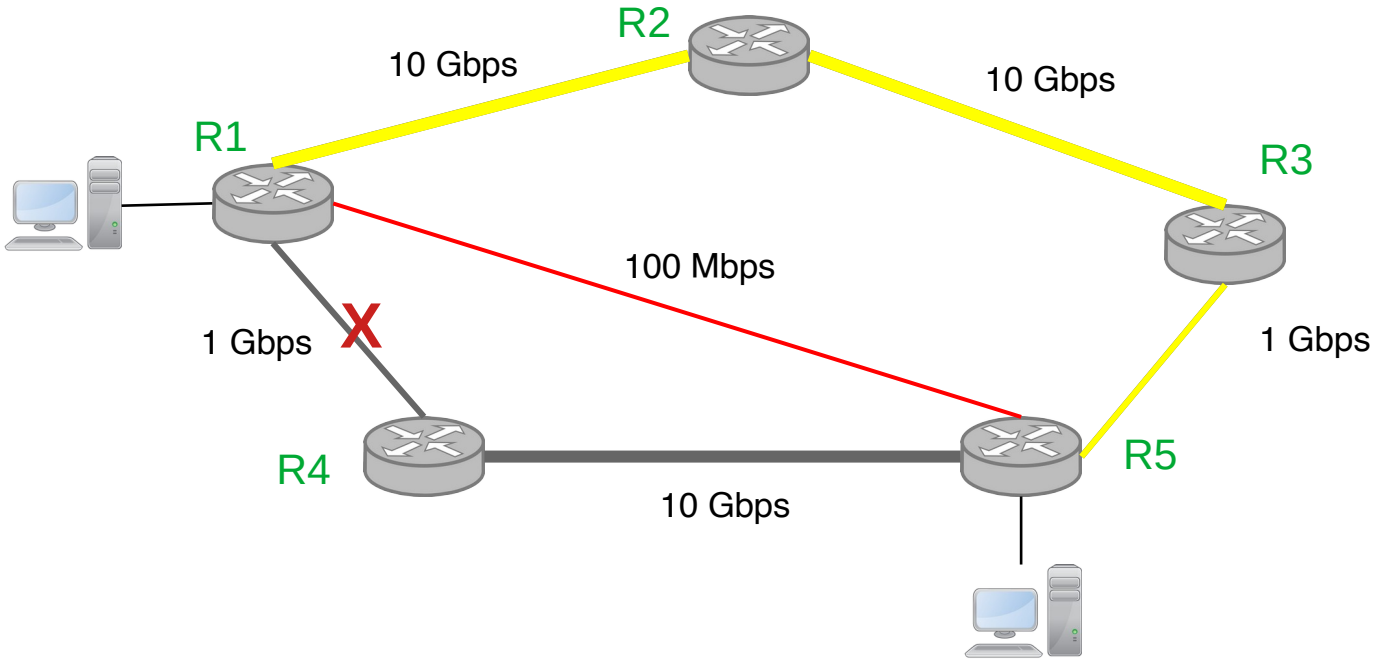
Distance-Vector Routing



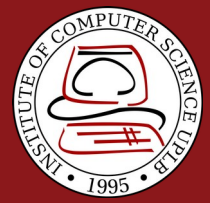
Link-State Routing



Path-Vector Routing



Policy
R1 and R4 not allowed



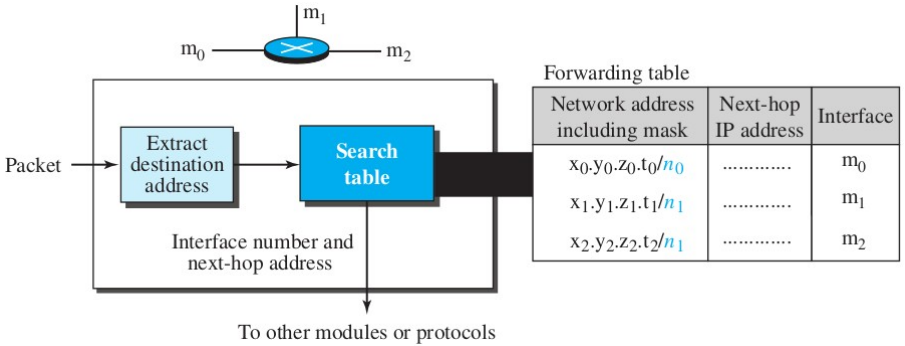
IP Forwarding

IP Forwarding

- Forwarding of packets in the Internet means delivering the packet to the next hop (can be the final destination host or a router)
- When IP is used as connection-less
  - Forwarding is based on the destination address of the IP datagram
- When IP is used as connection-oriented (virtual circuit packet switching)
  - Forwarding is based on the label attached to the IP datagram

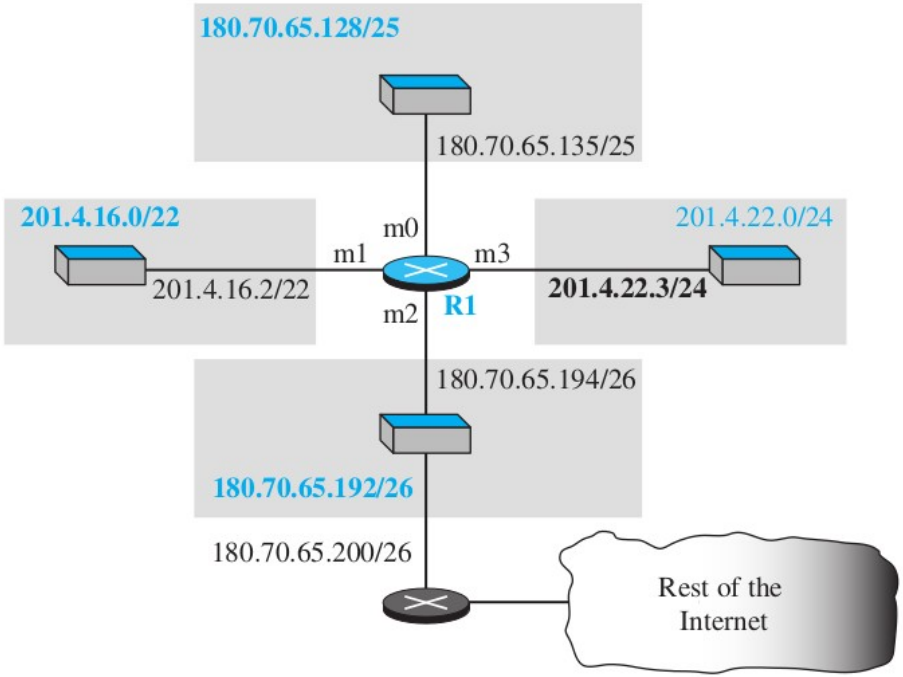
Forwarding based on Destination Address

- The traditional approach which requires a host or router to have a *forwarding/routing table*
- In classless addressing, the routing table must have the following information: network mask, network address, interface number, IP address of next router
- Forwarding module searches the forwarding table *row by row*



Forwarding based on Destination Address

- Example: Make the forwarding table for router R1 given the following configuration



Network address/mask	*Next hop*	*Interface*
180.70.65.192/**26**	—	m2
180.70.65.128/**25**	—	m0
201.4.22.0/**24**	—	m3
201.4.16.0/**22**	—	m1
Default	180.70.65.200	m2

Forwarding based on Destination Address

- Example: Show the forwarding process if a packet arrives at R1 with the destination address 180.70.65.140
- Solution: Process row by row to get the network address from the destination address
  - First Row: 180.70.65.140 & 255.255.255.192 = 180.70.65.128 (Not a match)
  - Second Row: 180.70.65.140 & 255.255.255.128 = 180.70.65.128 (A match!), thus forward to interface m0

Network address/mask	*Next hop*	*Interface*
180.70.65.192/**26**	—	m2
180.70.65.128/**25**	—	m0
201.4.22.0/**24**	—	m3
201.4.16.0/**22**	—	m1
Default	180.70.65.200	m2

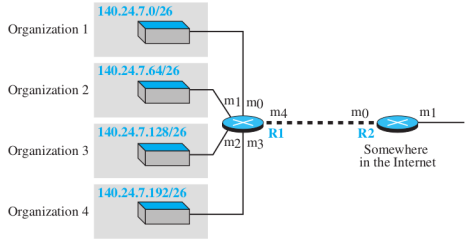
Forwarding Table Search Algorithm

- *Longest Prefix Match*: forwarding table is sorted from the longest mask to the shortest mask
  - Example: If there are three masks: /27, /26, /24 then /27 must be the first entry

- It is possible that the forwarding table may grow very large thus the algorithm may become very slow
- Solution is to use a different data structure, such as a *trie*

Other Approaches (Destination Address based)

- Address Aggregation
  - Blocks of addresses are aggregated into one larger block



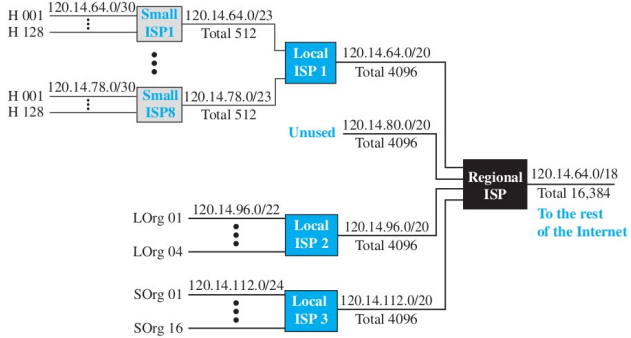
Forwarding table for R1

Network address/mask	Next-hop address	Interface
140.24.7.0/26	----------	m0
140.24.7.64/26	----------	m1
140.24.7.128/26	----------	m2
140.24.7.192/26	----------	m3
0.0.0.0/0	address of R2	m4

Forwarding table for R2

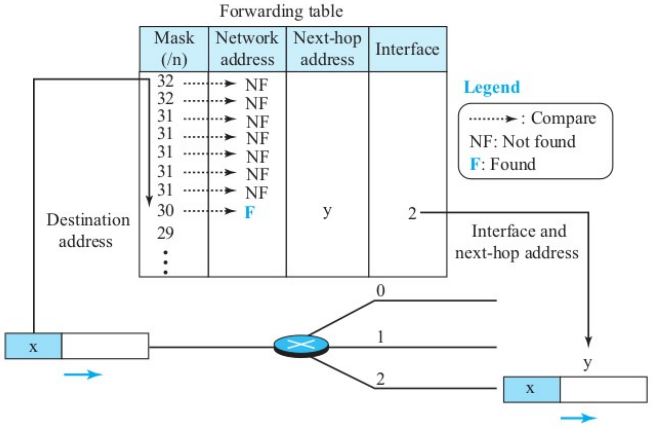
Network address/mask	Next-hop address	Interface
140.24.7.0/24	----------	m0
0.0.0.0/0	default router	m1

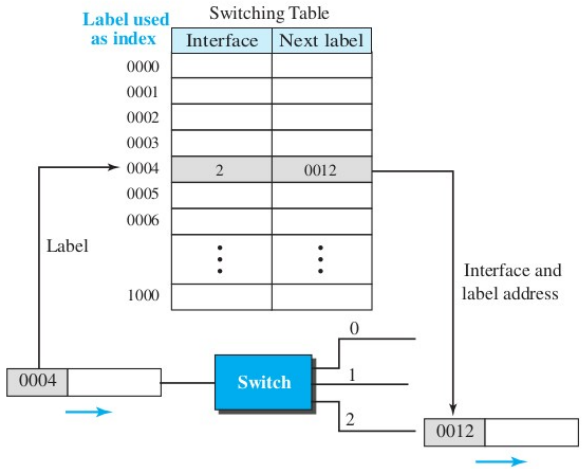
- Hierarchical Routing and Geographical Routing
  - Takes advantage of the hierarchy in the Internet



Forwarding based on Label

- Used in connection-oriented IP (virtual circuit packet switching)
- Intermediate node is called *switch*
- The switch forwards packet based on the label on the packet
- Uses *indexing* instead of searching



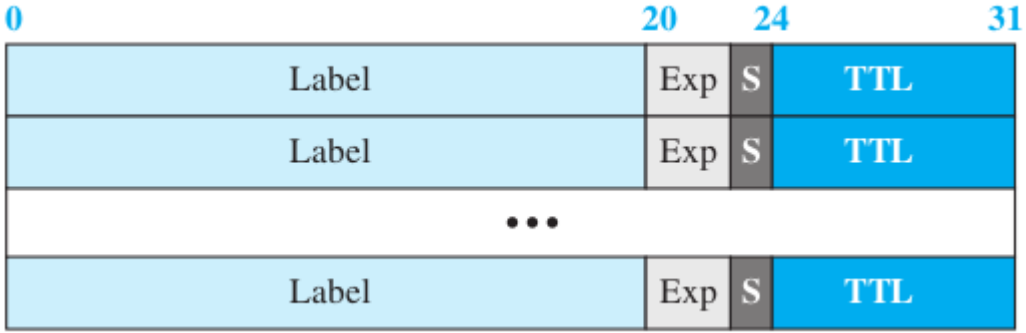


Forwarding based on Label: MPLS

- *Multi-Protocol Label Switching*
- Intermediate nodes can behave like a router or a switch
- IP datagram encapsulated in an MPLS header to provide the *stack of labels*
- Stacking of labels allow for hierarchical switching

Forwarding based on Label: MPLS Header

- *Label* – 20-bit field used to index the switching table
- *Exp* – 3-bit field reserved for experimental purposes
- *S* – 1-bit field which when set to 1 means that the header is the last one in the stack
- *TTL* – 8-bit field for Time To Live similar to IP header field



References

- Forouzan, B.A. 2013. Data Communications and Networking, 5th Ed. McGraw-Hill, New York.